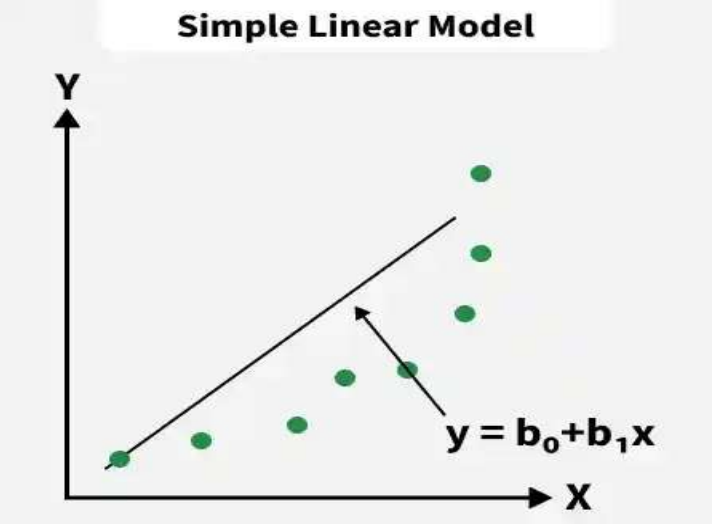
Day – 10 Regression

Regression in Machine Learning:

Regression is a supervised learning technique used to predict continuous numerical values by learning relationships between input variables (features) and an output variable (target). It helps understand how changes in one or more factors influence a measurable outcome and is widely used in forecasting, risk analysis, decision-making and trend estimation.



Linear Regression in Machine learning

Linear Regression is a fundamental supervised learning algorithm used to model the relationship between a dependent variable and one or more independent variables. It predicts continuous values by fitting a straight line that best represents the data.

- It assumes that there is a linear relationship between the input and output
- Uses a best-fit line to make predictions
- Commonly used in forecasting, trend analysis, and predictive modelling

For example, suppose we want to predict a student's exam score based on the number of hours studied. As study hours increase, exam scores generally increase as well. Here:

- **Independent variable (input):** Hours studied because it's the factor we control or observe.
- **Dependent variable (output):** Exam score because it depends on how many hours were studied.

Linear regression uses the independent variable to predict the dependent variable.

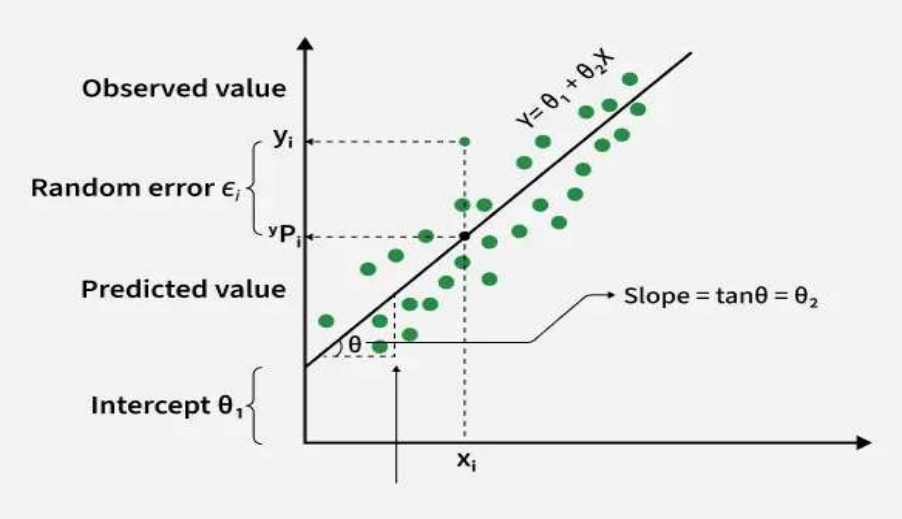
Best Fit Line

In linear regression, the best-fit line is the straight line that best represents the relationship between the independent variable (input) and the dependent variable (output). The goal is to

minimize the difference between the actual data points and the predicted values generated by the model.

1. Goal of the Best-Fit Line

The goal of linear regression is to find a straight line that minimizes the error (the difference) between the observed data points and the predicted values. This line helps us predict the dependent variable for new, unseen data.



Here Y is called a dependent or target variable and X is called an independent variable also known as the predictor of Y.

- θ_1 represents the intercept, which is the value of Y when X = 0
- θ_2 represents the slope, which shows how much Y changes for a unit change in X

There are many types of functions or modules that can be used for regression. A linear function is the simplest type of function. Here, X may be a single feature or multiple features representing the problem.

2. Equation of the Best-Fit Line

For simple linear regression (with one independent variable), the best-fit line is represented by the equation

$$y = mx + b$$

Where:

- **y** is the predicted value (dependent variable)
- **x** is the input (independent variable)
- **m** is the slope of the line (how much y changes when x changes)
- **b** is the intercept (the value of y when x = 0)

The best-fit line will be the one that optimizes the values of m (slope) and b (intercept) so that the predicted y values are as close as possible to the actual data points.

3. Minimizing the Error using the Least Squares Method

To determine the best-fit line, linear regression uses the Least Squares Method, which minimizes the difference between actual and predicted values. These differences are called residuals. These differences are called residuals.

The formula for residuals is:

$Residual = y_i - \hat{y}_i$

Where:

- y_i is the actual observed value
- $\hat{y}_i$ is the predicted value from the line for that x_i

The least squares method minimizes the sum of the squared residuals:

$$\Sigma(y_i - \hat{y}_i)^2$$

This method ensures that the line best represents the data where the sum of the squared differences between the predicted values and actual values is as small as possible.

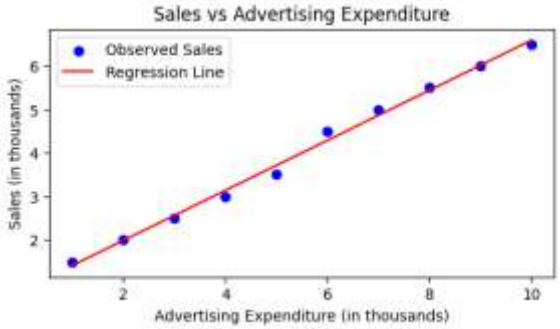
4. Interpretation of the Best-Fit Line

- **Slope (m):** The slope indicates how much the dependent variable changes for every one-unit increase in the independent variable. For example, if the slope is 5, then y increases by 5 units for every 1-unit increase in x.
- **Intercept (b):** The intercept represents the predicted value of y when x = 0. It's the point where the line crosses the y-axis.

1.Simple Linear Regression in Python

Simple linear regression models the relationship between a dependent variable and a single independent variable.

Simple Linear Regression aims to describe how one variable i.e the dependent variable changes in relation with reference to the independent variable. For example consider a scenario where a company wants to predict sales based on advertising expenditure. By using simple linear regression the company can determine if an increase in advertising leads to higher sales or not.



The relationship between the dependent and independent variables is represented by the simple linear equation:

$$y = mx + b$$

Here:

- y is the predicted value (dependent variable).
- m is the slope of the line
- x is the independent variable.
- b is the y-intercept (the value of y when x is 0).

To better understand this relationship we can express it in a more statistical context using the linear regression formula:

- $Y = \beta_0 + \beta_1 x$
- In simple linear regression the parameters β_0 and β_1 play crucial roles in defining the relationship between the independent variable X and the dependent variable Y in the regression equation.

Multiple Linear Regression using Python - ML

Linear regression is a statistical method used for predictive analysis. It models the relationship between a dependent variable and one or more independent variables by fitting a linear equation to the data. Multiple Linear Regression specifically extends this concept to include two or more independent variables.

Steps for Multiple Linear Regression

Steps to perform multiple linear regression are similar to that of simple linear Regression but difference comes in the evaluation process. We can use it to find out which factor has the highest influence on the predicted output and how different variables are related to each other. Equation for multiple linear regression is:

$$y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_n X_n$$

Where:

- $y$is the dependent variable
- $X_1, X_2, \cdots X_n$ are the independent variables
- β_0 is the intercept
- $\beta_1, \beta_2, \cdots \beta_n$ are the slopes

The goal of the algorithm is to find the best fit line equation that can predict the values based on the independent variables. A regression model learns from the dataset with known X and y values and uses it to predict y values for unknown X.

Applications

- **Predicting prices:** Used to predict the price of a house based on its size, location and other features.
- **Forecasting trends:** Model to forecast the sales of a product based on historical sales data.
- **Identifying risk factors:** Used to identify risk factors for heart patient based on patient medical data.
- **Making decisions:** It could be used to recommend which stock to buy based on market data.

Implementation:

Step 1 — Open Google Colab

Go to Google Colab and create:

Step 2 — First cell

We'll initially use a dataset directly from sklearn, so you don't need to download anything.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
# Load dataset
data = load_diabetes()
# Convert to DataFrame
df = pd.DataFrame(data.data, columns=data.feature_names)
# Add target column
df["target"] = data.target
# Display first 5 rows
df.head()
```

	age	sex	bmi	bp	s1	s2	s3	s4	s5	s6	target
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	-0.002592	0.019907	-0.017646	151.0
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412	-0.039493	-0.068332	-0.092204	75.0
2	0.085299	0.050680	0.044451	-0.005670	-0.045599	-0.034194	-0.032356	-0.002592	0.002861	-0.025930	141.0
3	-0.089063	-0.044642	-0.011595	-0.036656	0.012191	0.024991	-0.036038	0.034309	0.022688	-0.009362	206.0
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	-0.002592	-0.031988	-0.046641	135.0

Step 3 — Understand the dataset

```
print("Shape:", df.shape)
print("\nColumns:")
print(df.columns)
print("\nInformation:")
df.info()
print("\nStatistics:")
df.describe()
```

```
Shape: (442, 11)

Columns:
Index(['age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6',
       'target'],
      dtype='object')

Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 442 entries, 0 to 441
Data columns (total 11 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   age     442 non-null    float64
 1   sex     442 non-null    float64
```

Step 4 — Check the features

```
df.corr()["target"].sort_values(ascending=False)
```

	target
target	1.000000
bmi	0.586450
s5	0.565883
bp	0.441482
s4	0.430453
s6	0.382483
s1	0.212022
age	0.187889
s2	0.174054
sex	0.043062
s3	-0.394789

Step 5 — Choose X and y

We'll use BMI as the input feature and target as the value we want to predict.

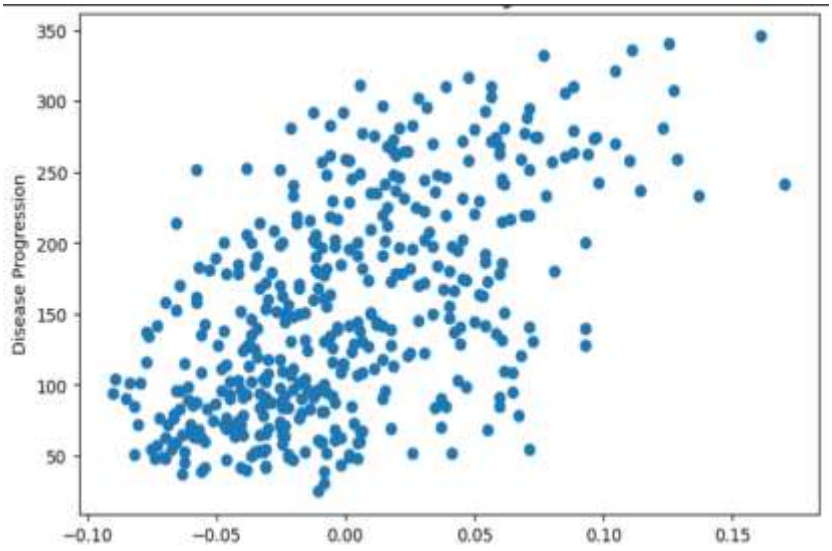
```
# Input feature
X = df[["bmi"]]
# Target variable
y = df["target"]
print("X shape:", X.shape)
print("y shape:", y.shape)
print("\nX:")
print(X.head())
print("\ny:")
print(y.head())
```

```
X shape: (442, 1)
y shape: (442,)

X:
        bmi
0  0.061696
1 -0.051474
2  0.044451
3 -0.011595
4 -0.036385
```

Step 6 — Visualize the data

```
plt.figure(figsize=(8, 5))
plt.scatter(X, y)
plt.xlabel("BMI")
plt.ylabel("Disease Progression")
plt.title("BMI vs Disease Progression")
plt.show()
```



Step 7 — Split the data

We divide the dataset into:

- **80% → Training data → model learns from this**
- **20% → Testing data → we use this to check the model**

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)

Training data: (353, 1)
Testing data: (89, 1)
```

Step 8 — Import and create the model

Step 9 — Train the model

Step 10 — See what the model learned

```
from sklearn.linear_model import LinearRegression
# Create the model
model = LinearRegression()
print(model)
model.fit(X_train, y_train)

LinearRegression()
 LinearRegression
LinearRegression()

print("Slope (Coefficient):", model.coef_[0])
print("Intercept:", model.intercept_)

Slope (Coefficient): 998.5776891375598
Intercept: 152.00335421448167
```

Step 11 — Make predictions 🔮

Now let's see what our model predicts for the test data.

This will show the first 10 predictions.

Then let's compare them with the actual values:

```
y_pred = model.predict(X_test)
print("Predictions:")
print(y_pred[:10])
comparison = pd.DataFrame({
    "Actual": y_test.values,
    "Predicted": y_pred
})
print(comparison.head(10))
```

```
Predictions:
[145.80622687 188.85739048 147.95878505 203.92529774 131.8145987
 127.50948234 322.31599764 197.4676232   61.85645785 167.33180868]
   Actual   Predicted
0   219.0  145.806227
1    70.0  188.857390
2   202.0  147.958785
3   230.0  203.925298
4   111.0  131.814599
5    84.0  127.509482
6   242.0  322.315998
7   272.0  197.467623
8    94.0   61.856458
9    96.0  167.331809
```

Step 12 — Evaluate the model 📊

We'll calculate four common metrics:

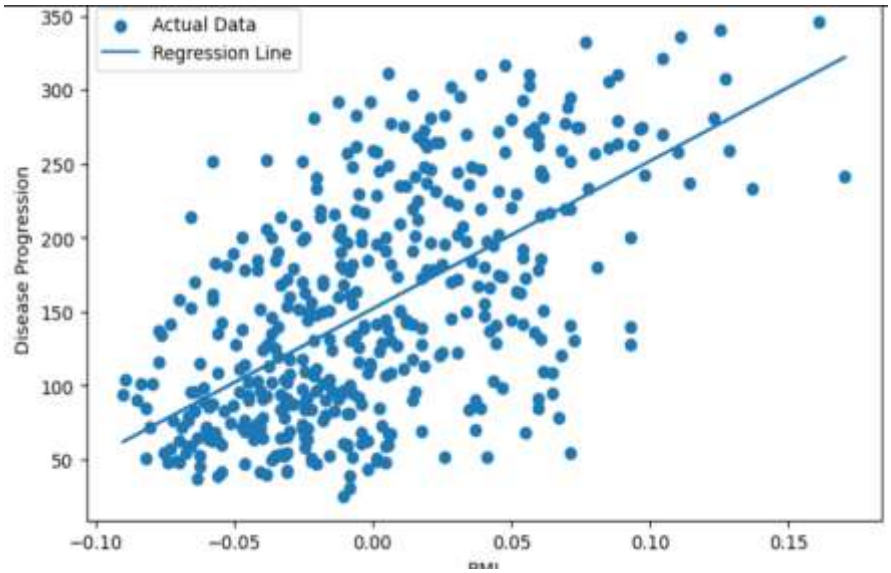
- **MAE — Mean Absolute Error**
- **MSE — Mean Squared Error**
- **RMSE — Root Mean Squared Error**
- **R² Score — how well the model explains the variation in the target**

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
# Calculate metrics
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
print("MAE :", mae)
print("MSE :", mse)
print("RMSE:", rmse)
print("R²   :", r2)
```

```
MAE : 52.259976445345536
MSE : 4061.8259284949268
RMSE: 63.73245584860925
R²   : 0.23335039815872138
```

Step 13 — Plot the regression line 📈

```
plt.figure(figsize=(8, 5))
# Actual data points
plt.scatter(X, y, label="Actual Data")
# Regression line
plt.plot(X, model.predict(X), label="Regression Line")
plt.xlabel("BMI")
plt.ylabel("Disease Progression")
plt.title("Linear Regression: BMI vs Disease Progression")
plt.legend()
plt.show()
```




```
new_bmi = [[0.05]]
prediction = model.predict(new_bmi)
print("New BMI:", new_bmi[0][0])
print("Predicted Disease Progression:", prediction[0])

New BMI: 0.05
Predicted Disease Progression: 201.93223867135867
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but LinearRegre
  warnings.warn(
```